Exception Handling in PYTHON

1. Exception Handling in Python

An exception is an error that occurs during the execution of a program and interrupts its normal flow.

For example, dividing a number by zero causes a ZeroDivisionError in Python.

Exception handling allows us to detect and handle errors so that the program does not terminate unexpectedly.

Common exceptions in Python

Exception	Meaning
ZeroDivisionError	Division by zero
ValueError	Invalid value supplied to an operation
TypeError	Operation performed on an inappropriate data type
NameError	Variable is not defined
FileNotFoundError	Requested file does not exist
IndexError	List index is out of range

2. The try and except blocks

- try block: Contains code that may generate an exception.
- except block: Contains code that handles the exception.

Syntax:

```
try:
    # Code that may cause an exception
    statements
except ExceptionType:
    # Code to handle the exception
    Statements
```

WAP to divide two numbers when the denominator is zero

Problem statement: Write a Python program to divide two numbers. If the denominator is zero, handle the exception using `try` and `except`.

```
try:
    a = int(input("Enter numerator: "))
    b = int(input("Enter denominator: "))
```

```
    result = a / b
    print("Result =", result)

except ZeroDivisionError:
    print("Error: Division by zero is not allowed.")
```

3. Using multiple `except` blocks

A program may encounter different types of exceptions. We can use multiple `except` blocks to handle them separately.

```
try:
    a = int(input("Enter numerator: "))
    b = int(input("Enter denominator: "))

    print("Result =", a / b)

except ZeroDivisionError:
    print("Error: Cannot divide by zero.")

except ValueError:
    print("Error: Please enter valid integers.")
```

Here, `ZeroDivisionError` handles division by zero, while `ValueError` handles input such as `abc` when an integer is expected.

4. The `else` and `finally` blocks

Python provides two additional blocks for exception handling.

- `else`: Executes only when no exception occurs in the `try` block.
- `finally`: Executes whether an exception occurs or not, making it useful for cleanup operations.

```
try:
    a = int(input("Enter numerator: "))
    b = int(input("Enter denominator: "))
    result = a / b
```

```
except ZeroDivisionError:
    print("Cannot divide by zero.")

except ValueError:
    print("Invalid input.")

else:
    print("Result =", result)

finally:
    print("Program execution completed.")
```

PRACTICAL 9: USER DEFINED EXCEPTIONS

Definition: A user-defined exception is an exception created by the programmer to handle specific error conditions according to the requirements of a program.

Examples:

- Checking whether a student has passed or failed.
- Checking whether a person is eligible to vote.
- Checking whether a bank account has sufficient balance.
- Checking whether marks are within the valid range.

Syntax for creating a user-defined exception

A custom exception is generally created by defining a class that inherits from Python's built-in `Exception` class.

```
class MyException(Exception):
    pass
```

To raise the exception, we use the `raise` keyword.

```
raise MyException("This is a custom exception")
```

Important keywords

Keyword	Description
`class`	Defines a class.
`Exception`	Base class for exceptions in Python.
`pass`	Represents an empty class or block.
`raise`	Raises an exception explicitly.
`try`	Contains statements that may generate an exception.
`except`	Handles the exception.

`as e` Stores the exception object in a variable named `e`

WAP to create your own exception

Problem statement: Write a Python program to create a user-defined exception that checks whether a person is eligible to vote. A person must be at least 18 years old to vote.

```
class InvalidAgeError(Exception):
    pass

try:
    age = int(input("Enter your age: "))

    if age < 18:
        raise InvalidAgeError(
            "You are not eligible to vote."
        )

    print("You are eligible to vote.")

except InvalidAgeError as e:
    print("Custom Exception:", e)

except ValueError:
    print("Please enter a valid integer.")
```

WAP to create a custom exception for negative numbers

Problem statement: Write a Python program that raises a custom exception if the user enters a negative number.

```
class NegativeNumberError(Exception):
    pass

try:
    num = int(input("Enter a number: "))

    if num < 0:
        raise NegativeNumberError(
            "Negative numbers are not allowed."
        )
```

```
    print("You entered:", num)

except NegativeNumberError as e:
    print("Custom Exception:", e)

except ValueError:
    print("Please enter a valid integer.")
```